<u>Lab Report:</u> Convolutional Neural Networks — CIFAR-10

<u>Course:</u> BFOR516 - Advanced Data Analytics for Cyber

<u>Student:</u> Spencer Kone

<u>Date:</u> March 2026

<u>AI Usage Statement</u>

I used Antigravity and Claude to assist with this assignment. It was helpful in coming up with ideas and in phrasing and expressing content. All analysis, interpretation and conclusions were done by Spencer Kone.

<u>1. Objective</u>

Objective: Implement and improve a CNN model to classify images from the CIFAR-10 dataset into their respective classes.

This assignment is about the effect of design decisions on the performance of a model. For this lab you need to:

Compare a baseline CNN architecture with an alternate implementation

Investigate how different architectural choices affect model accuracy and generalization

Use the trained model to classify images in the test dataset

This section shows how changes to the graph structure impact the learning dynamics and the accuracy of the model.

2. Dataset

CIFAR-10 is an open source collection of 60,000 32 x 32 color training images in 10 classes. The dataset was originally created by the Canadian Institute for Advanced Research (CIFAR). It is one of the simplest and most commonly used benchmarks for deep learning image classification.

Property	Value
Total images	60,000
Image dimensions	32 × 32 pixels, 3 channels (RGB)
Classes	10 (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)
Training set	50,000 images
Test set	10,000 images
Samples per class (test)	1,000 each (balanced)

The dataset is directly loaded using tf.keras.datasets.cifar10.

Pixel values must be normalized before training. The original pixel values were in the range of [0,255] so I just divided by 255 to normalize it to the range of [0.0,1.0] that is a recommended range for deep learning network training.

The data set has already been split, is balanced in terms of the classes and has not been corrupted with any missing values.

3. Model Architectures

In this experiment we have developed two CNNs. Both networks are trained under the same circumstances.

Optimizer: Adam

Loss function: Sparse categorical cross-entropy

Epochs: 10

Batch size: 64

Validation split: 10%

By applying the design of the network in the figure, we can have approximately the same number of parameters in both models.

Model 1 — Baseline CNN

This is a baseline model which closely resembles the minimal model described in the "Lab instructions" page. The baseline model is similar to the minimal model but includes two additional convolutional layers with pooling in between. Following these layers is a flattening layer and a dense layer for classification output.

Input: (32, 32, 3)

Conv2D(32, 3×3, relu, valid) → (30, 30, 32)

MaxPooling2D(2×2) → (15, 15, 32)

Conv2D(64, 3×3, relu, valid) → (13, 13, 64)

MaxPooling2D(2×2) → (6, 6, 64)

Flatten() → 2,304

Dense(128, relu)

Dense(10, softmax)

This model does not include padding, batch normalization, or other regularization techniques.

Total parameters: 315,722 (1.20 MB)

Model 2 — Improved CNN

Second, a deep network model is proposed to deal with the overfitting problem and the low-level feature that is not fully explored in the baseline network.

Several modifications were introduced:

Convolution blocks were deepened

Batch normalization layers were added

Dropout regularization was introduced

Padding was changed to preserve spatial resolution

Input: (32, 32, 3)

Block 1

Conv2D(32, 3×3, same, relu) → BatchNorm

Conv2D(32, 3×3, same, relu) → BatchNorm

MaxPool(2×2) → Dropout

Block 2

Conv2D(64, 3×3, same, relu) → BatchNorm

Conv2D(64, 3×3, same, relu) → BatchNorm

MaxPool(2×2) → Dropout

Flatten() → 4,096

Dense(256, relu) → BatchNorm → Dropout(0.5)

Dense(10, softmax)

Total parameters: 1,118,762 (4.27 MB)

Design Changes and Rationale

Change Rationale

When set to same it ensures that spatial dimensions remain the same before we apply a pooling layer. Without padding the dimension of each convolution will be reduced.

Convolutional blocks with double convolutional layers VGG-like architecture, with two convolutional layers in each block in order to exploit the complex features before spatial information is lost.

BatchNormalization Normalizes intermediate activations to stabilize training and improve gradient flow.

Dropout is a technique that prevents overfitting by forcing the network to learn in a highly decentralized way. If a neural network is learning the training set too well then it may be overfitting to the data. One of the most popular techniques for preventing overfitting is dropout. When using dropout, training data is fed into the network. For each forward and backward pass through the network a random subset of the neurons are set to zero.

Larger dense layer The output from a deeper convolution stack is usually a higher dimensional feature set and this calls for a wider dense layer.

4. Training Results

Accuracy and Loss Summary

Metric	Model 1	Model 2
Test Accuracy	66.21%	78.37%
Test Loss	1.0848	0.6404
Final Train Accuracy	80.45%	82.58%
Final Validation Accuracy	68.18%	79.98%
Parameters	315,722	1,118,762

Model 2 achieves 12.16% higher test accuracy than the baseline.

Training Behavior Analysis

There are definitely some signs of overfitting in the baseline model. The training accuracy increases in a smooth, predictable fashion and levels off around 80%. The validation accuracy increases and then drops. The validation loss is also starting to rise. Overfitting can occur when the model is learning the noise in the training data as opposed to learning the actual patterns.

Training behavior has improved. The training and validation accuracy are almost identical after ten epochs of training. Additionally, the validation loss is still decreasing, so the model has not yet started to overfit the noise.

The introduction of BatchNormalization and Dropout significantly improved generalization performance.

Per-Class Performance (Model 2)

Class	Precision	Recall	F1	Notes
automobile	0.92	0.87	0.89	Distinct vehicle shape
ship	0.84	0.91	0.88	Strong ocean background cue
truck	0.85	0.90	0.87	Similar features to automobiles
frog	0.86	0.79	0.83	Distinct body structure
horse	0.78	0.87	0.82	Clear silhouette
airplane	0.84	0.81	0.82	Recognizable outline
deer	0.78	0.74	0.76	Some confusion with horse
bird	0.83	0.60	0.70	Low recall
dog	0.59	0.81	0.68	High recall but lower precision
cat	0.62	0.54	0.58	Most difficult class

One more observation that is worth mentioning is the well known cat-dog confusion observed in the CIFAR-10 data. We have that about 26.7% of the true cats are classified as dogs in Model 2.

This is really tough. I think one of the major factors here is that our images are 32x32 pixels. At this scale, the texture and visual properties of cats and dogs are often very similar. This makes it very difficult for the network to learn discrimination between the two classes.

Bird images also have low recall scores, possibly due to large pose and size variation within the dataset.

Confusion Matrix Observations

Baseline model

Airplane → Ship: 140 misclassifications

Cat → Dog: 207 misclassifications

Bird → Dog: 141 misclassifications

Improved model

Cat → Dog: 267 misclassifications

Bird → Dog: 96 misclassifications

Adding padding reduced the number of mistakes between the target and attacking aircraft from 140 to 64. The padding was added to the convolution layer to preserve the spatial details of the input image so that the visual shape features of the objects could be retained, especially for those objects that have similar shapes.

5. Sample Predictions

We chose to test 20 images from the test set.

Model 2 achieved an accuracy of 17/20 = 85% on this subset, which is only a few percentage points less than the estimated 78% accuracy the model achieved on the test set.

False alarms are commonly generated by small animals because they are easily triggered by camera views at a distant range, and also often appear in a limited area of the frame.

6. Conclusion

This experiment demonstrates how architectural design decisions influence CNN performance.

Several key insights emerged:

Regularization is necessary for generalization.

If you remove the regularization from the baseline model, you will see how the model overfits the training set. Adding dropout and batch normalization helps to narrow the gap.

Deeper convolution blocks improve feature learning.

This layer increases the number of features the network can learn from the input. Stacking convolutional layers in this way enables the network to learn more complex features before the spatial information is reduced by pooling.

Preserving spatial dimensions improves classification.

Padding='same' is used to preserve the spatial information in the signal. This is particularly useful when we want to differentiate between objects of approximately the same size.

Some classification errors are unavoidable at low resolution.

Even with the classes we defined as easy to distinguish such as cats and dogs it is still quite hard to train a model to classify an image given a 32x32 pixel input. Perhaps with higher resolution images or more data augmentation.

After 10 epochs of training with CPU, the improved CNN model can reach 78.37% accuracy, which is still excellent performance for a model of this size.

Future improvements could include:

Increasing the number of training epochs

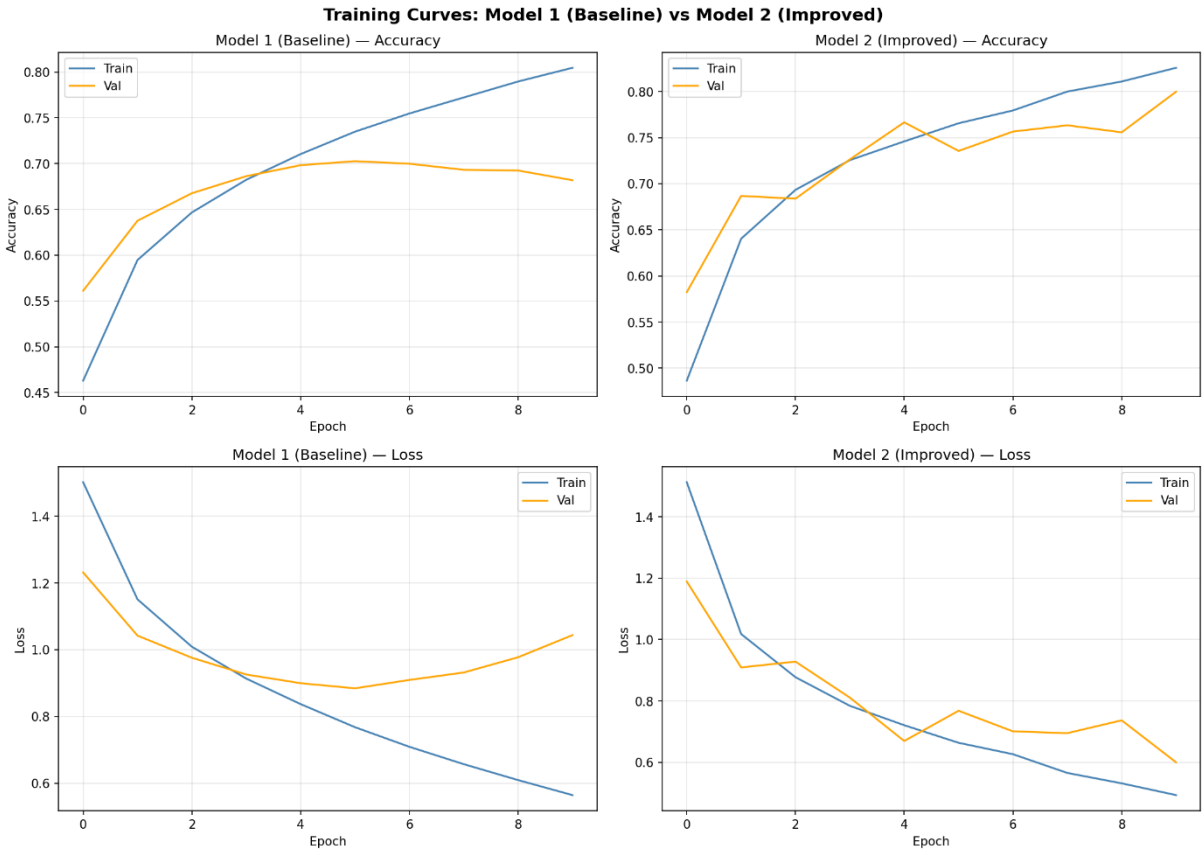
Applying data augmentation techniques such as random flips and crops

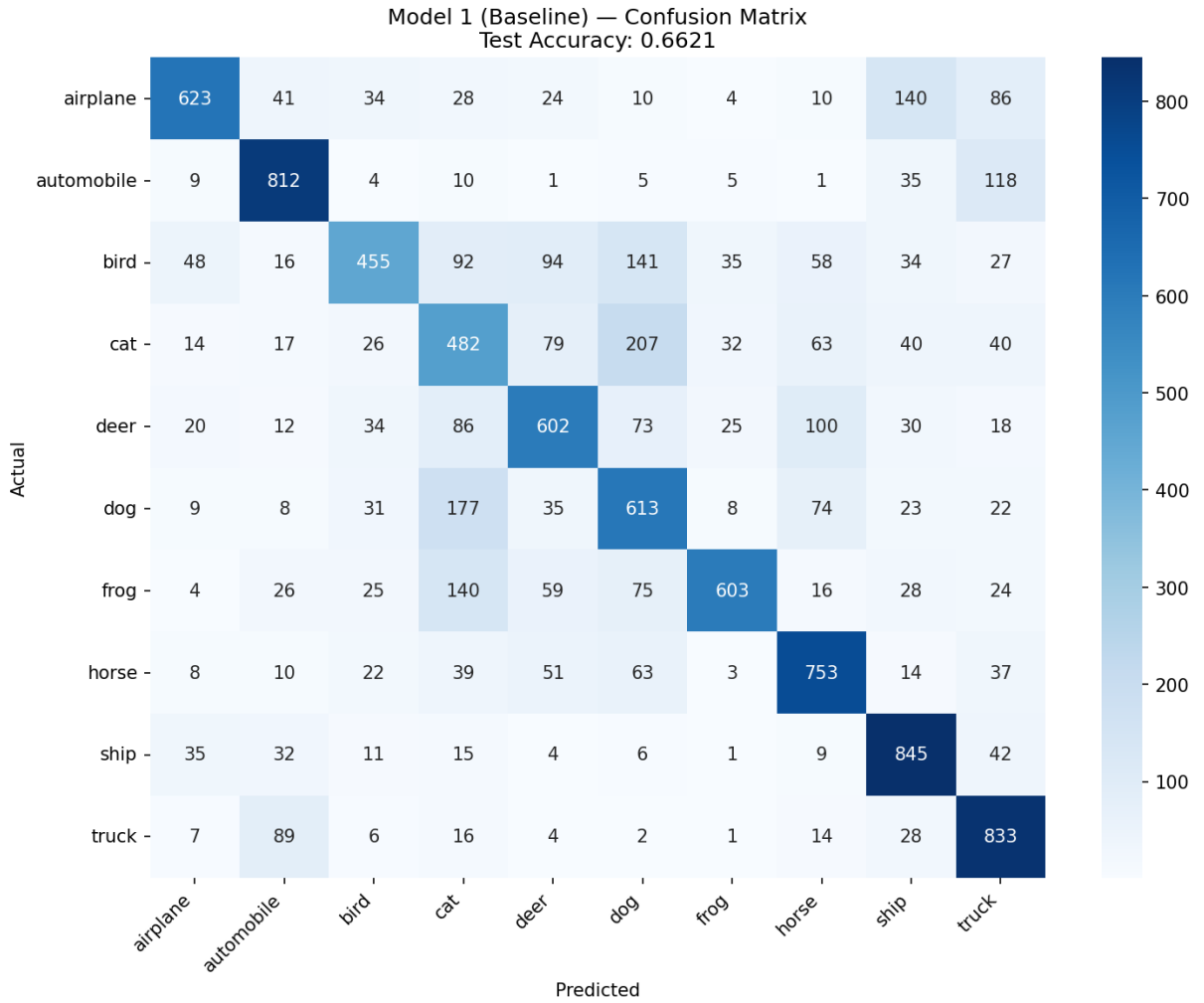
Adding additional convolution blocks

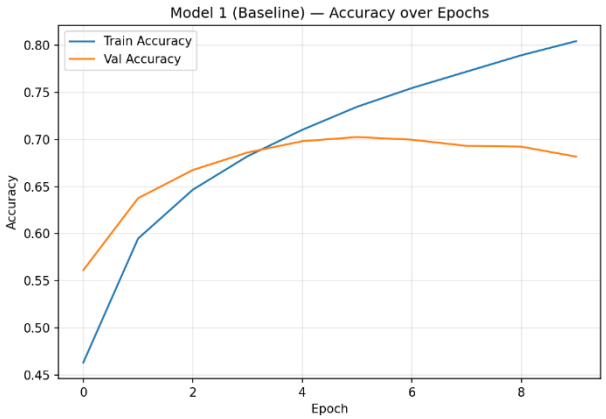
7. Output Files

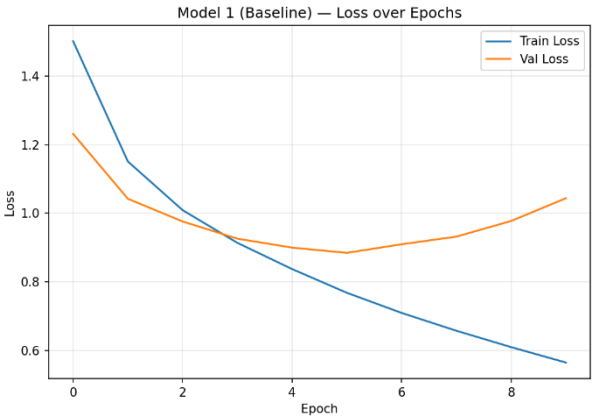
File	Description
model1_training_curves.png	Accuracy and loss curves for Model 1
model2_training_curves.png	Accuracy and loss curves for Model 2
comparison_training_curves.png	Side-by-side comparison of both models
model1_confusion_matrix.png	Confusion matrix for baseline model
model2_confusion_matrix.png	Confusion matrix for improved model

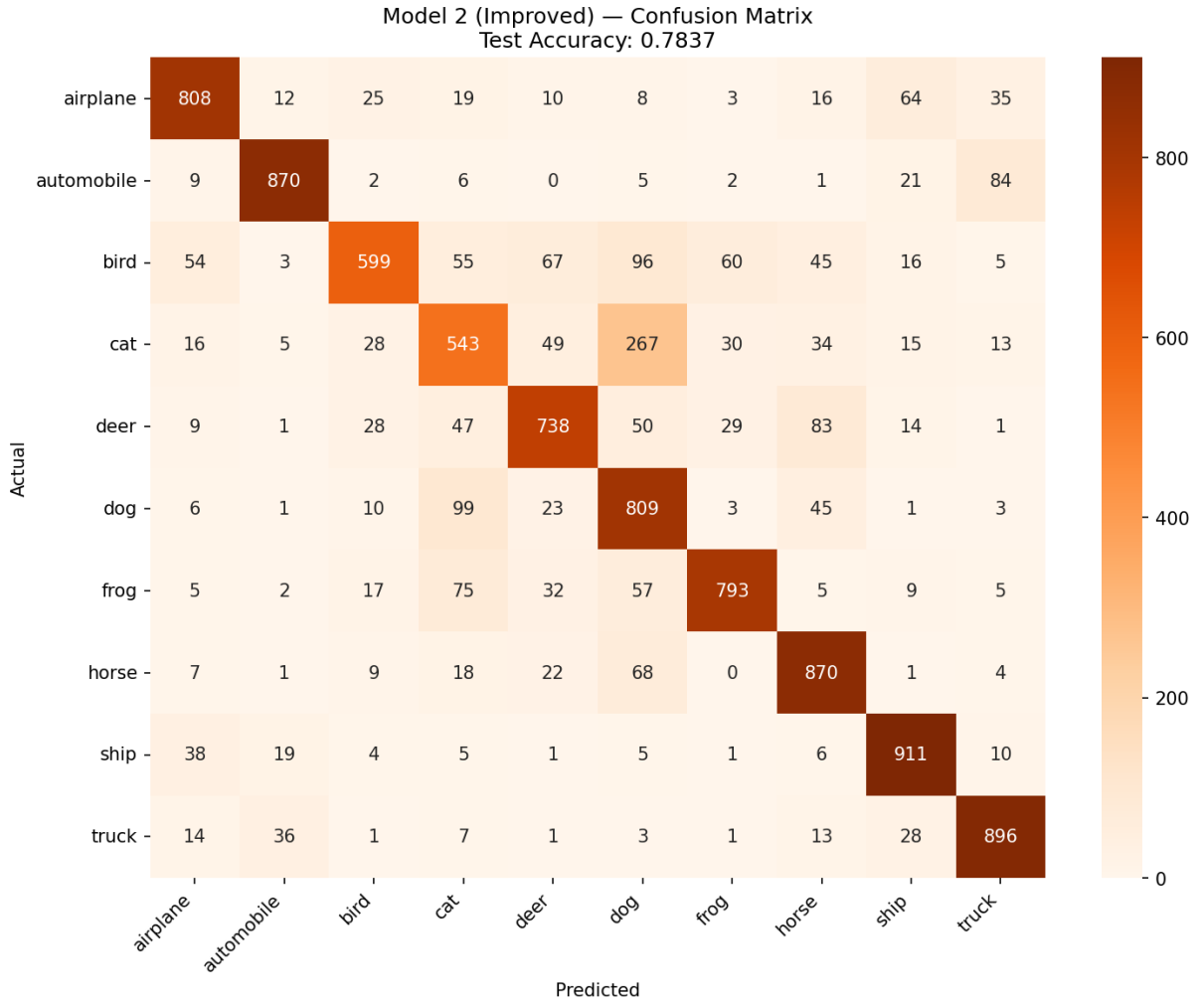
Visualization

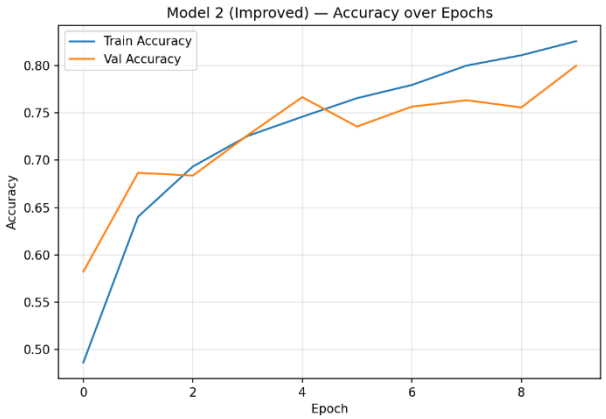

Training Curves: Model 1 (Baseline) vs Model 2 (Improved)
Model 1 (Baseline) — Accuracy
Model 2 (Improved) — Accuracy
Model 1 (Baseline) — Loss
Model 2 (Improved) — Loss
Train
Val
Accuracy
Loss
Epoch

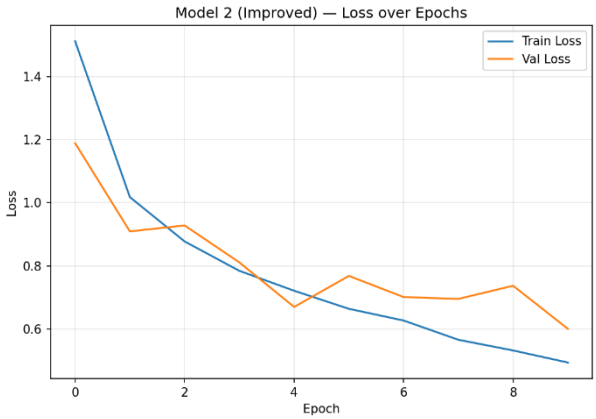

Model 1 (Baseline) — Confusion Matrix
Test Accuracy: 0.6621
Actual
Predicted
airplane 623 41 34 28 24 10 4 10 140 86
automobile 9 812 4 10 1 5 5 1 35 118
bird 48 16 455 92 94 141 35 58 34 27
cat 14 17 26 482 79 207 32 63 40 40
deer 20 12 34 86 602 73 25 100 30 18
dog 9 8 31 177 35 613 8 74 23 22
frog 4 26 25 140 59 75 603 16 28 24
horse 8 10 22 39 51 63 3 753 14 37
ship 35 32 11 15 4 6 1 9 845 42
truck 7 89 6 16 4 2 1 14 28 833
airplane
automobile
bird
cat
deer
dog
frog
horse
ship
truck
800
700
600
500
400
300
200
100

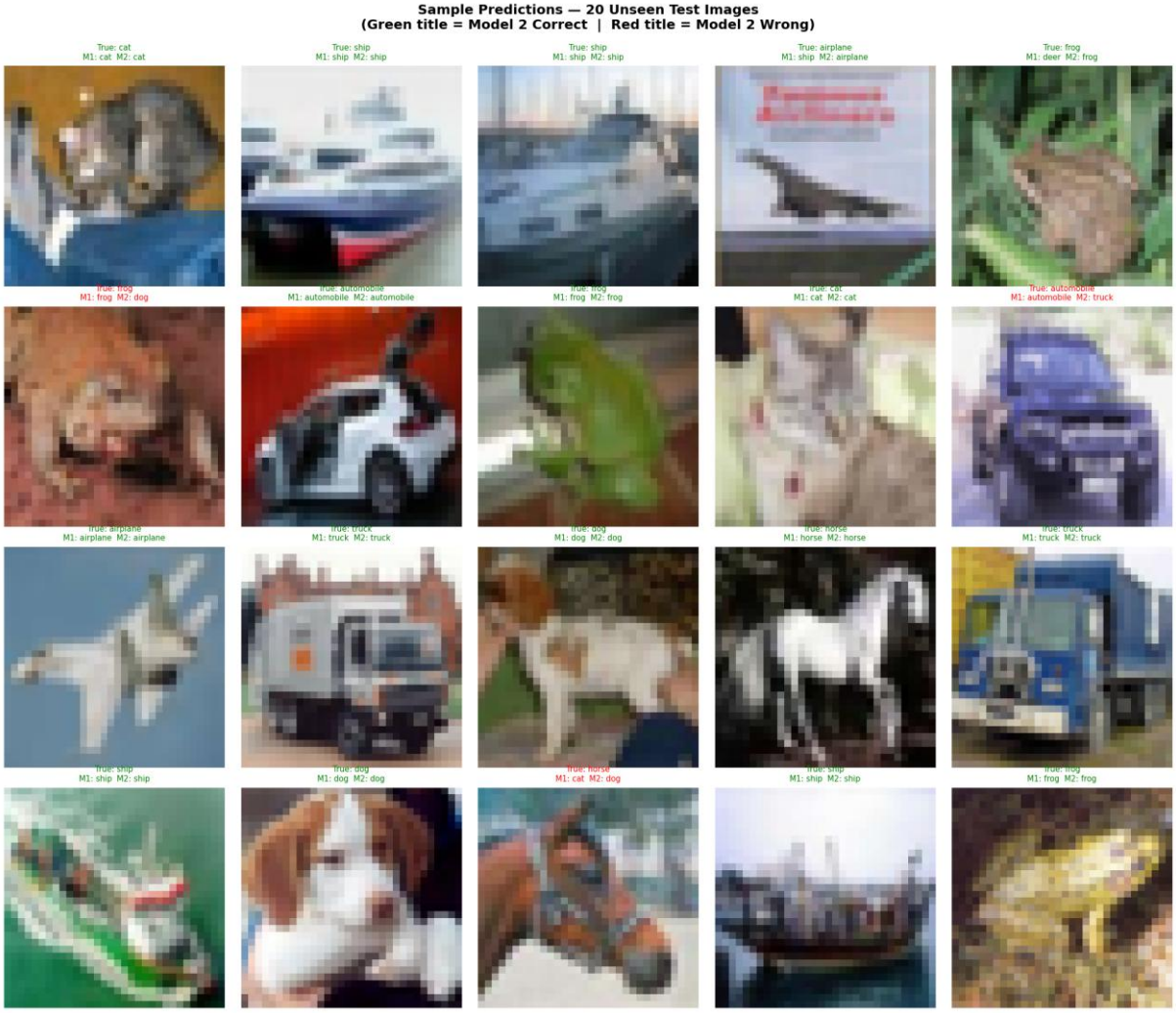

Model 1 (Baseline) — Accuracy over Epochs
Train Accuracy
Val Accuracy
Accuracy
Epoch
0.80
0.75
0.70
0.65
0.60
0.55
0.50
0.45
0
2
4
6
8


Model 1 (Baseline) — Loss over Epochs
Train Loss
Val Loss
Loss
Epoch
1.4
1.2
1.0
0.8
0.6
0
2
4
6
8


Model 2 (Improved) — Confusion Matrix
Test Accuracy: 0.7837
Actual
Predicted
airplane 808 12 25 19 10 8 3 16 64 35
automobile 9 870 2 6 0 5 2 1 21 84
bird 54 3 599 55 67 96 60 45 16 5
cat 16 5 28 543 49 267 30 34 15 13
deer 9 1 28 47 738 50 29 83 14 1
dog 6 1 10 99 23 809 3 45 1 3
frog 5 2 17 75 32 57 793 5 9 5
horse 7 1 9 18 22 68 0 870 1 4
ship 38 19 4 5 1 5 1 6 911 10
truck 14 36 1 7 1 3 1 13 28 896
airplane automobile bird cat deer dog frog horse ship truck
0 200 400 600 800


Model 2 (Improved) — Accuracy over Epochs
Train Accuracy
Val Accuracy
Accuracy
Epoch
0.50 0.55 0.60 0.65 0.70 0.75 0.80
0 2 4 6 8


Model 2 (Improved) — Loss over Epochs
Train Loss
Val Loss
Loss
Epoch
0.6 0.8 1.0 1.2 1.4
0 2 4 6 8


Sample Predictions — 20 Unseen Test Images
(Green title = Model 2 Correct | Red title = Model 2 Wrong)
True: cat
M1: cat M2: cat
True: ship
M1: ship M2: ship
True: ship
M1: ship M2: ship
True: airplane
M1: ship M2: airplane
True: frog
M1: deer M2: frog
True: frog
M1: frog M2: dog
True: automobile
M1: automobile M2: automobile
True: frog
M1: frog M2: frog
True: cat
M1: cat M2: cat
True: automobile
M1: automobile M2: truck
True: airplane
M1: airplane M2: airplane
True: truck
M1: truck M2: truck
True: dog
M1: dog M2: dog
True: horse
M1: horse M2: horse
True: truck
M1: truck M2: truck
True: ship
M1: ship M2: ship
True: dog
M1: dog M2: dog
True: horse
M1: cat M2: dog
True: ship
M1: ship M2: ship
True: frog
M1: frog M2: frog

Appendix — Full Console Output

```

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1773457467.544637   13976 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.

======================================================================

STEP 1: Loading and Preprocessing CIFAR-10

======================================================================

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz

170498071/170498071 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8s 0us/step

x_train shape: (50000, 32, 32, 3)  (samples, H, W, C)

x_test  shape: (10000, 32, 32, 3)

y_train shape: (50000, 1)  (samples, 1) — integer labels 0-9

Classes: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

Train samples: 50,000  |  Test samples: 10,000


Label value range: 0 – 9

Class counts in test set:

  0 (airplane    ): 1000

  1 (automobile  ): 1000

  2 (bird        ): 1000

  3 (cat         ): 1000

  4 (deer        ): 1000

  5 (dog         ): 1000

  6 (frog        ): 1000

  7 (horse       ): 1000

  8 (ship        ): 1000
```

9 (truck): 1000

==========

MODEL 1: Baseline CNN

==========

Architecture:

Conv2D(32, (3,3), relu) → (30,30,32)

MaxPooling2D(2,2) → (15,15,32)

Conv2D(64, (3,3), relu) → (13,13,64)

MaxPooling2D(2,2) → (6,6,64)

Flatten() → 2304

Dense(128, relu)

Dense(10, softmax)

I0000 00:00:1773457488.302644 13976 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Model: "Model1_Baseline"

┌──┐

│ Layer (type) │ Output Shape │ Param # │

├──┤

```
│ conv2d (Conv2D)                 │ (None, 30, 30, 32)      │           896 │
├─────────────────────────────────┼─────────────────────────┼───────────────┤
│ max_pooling2d (MaxPooling2D)    │ (None, 15, 15, 32)      │             0 │
├─────────────────────────────────┼─────────────────────────┼───────────────┤
│ conv2d_1 (Conv2D)               │ (None, 13, 13, 64)      │        18,496 │
├─────────────────────────────────┼─────────────────────────┼───────────────┤
│ max_pooling2d_1 (MaxPooling2D)  │ (None, 6, 6, 64)        │             0 │
├─────────────────────────────────┼─────────────────────────┼───────────────┤
│ flatten (Flatten)               │ (None, 2304)            │             0 │
├─────────────────────────────────┼─────────────────────────┼───────────────┤
│ dense (Dense)                   │ (None, 128)             │       295,040 │
├─────────────────────────────────┼─────────────────────────┼───────────────┤
│ dense_1 (Dense)                 │ (None, 10)              │         1,290 │
└─────────────────────────────────┴─────────────────────────┴───────────────┘
```

Total params: 315,722 (1.20 MB)

Trainable params: 315,722 (1.20 MB)

Non-trainable params: 0 (0.00 B)

Training Model 1 (epochs=10, batch_size=64, val_split=0.1)...

Epoch 1/10

704/704 ━━━━━━━━━━━━━━━━━━━━ 7s 7ms/step - accuracy: 0.4630 - loss: 1.5019 - val_accuracy: 0.5612 - val_loss: 1.2314

Epoch 2/10

```
704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.5947
- loss: 1.1509 - val_accuracy: 0.6376 - val_loss: 1.0421

Epoch 3/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.6466
- loss: 1.0089 - val_accuracy: 0.6676 - val_loss: 0.9759

Epoch 4/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.6823
- loss: 0.9135 - val_accuracy: 0.6862 - val_loss: 0.9257

Epoch 5/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.7103
- loss: 0.8370 - val_accuracy: 0.6982 - val_loss: 0.8996

Epoch 6/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.7348
- loss: 0.7678 - val_accuracy: 0.7026 - val_loss: 0.8842

Epoch 7/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.7546
- loss: 0.7091 - val_accuracy: 0.6998 - val_loss: 0.9094

Epoch 8/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.7721
- loss: 0.6571 - val_accuracy: 0.6932 - val_loss: 0.9316

Epoch 9/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.7895
- loss: 0.6093 - val_accuracy: 0.6924 - val_loss: 0.9773

Epoch 10/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - accuracy: 0.8045
- loss: 0.5644 - val_accuracy: 0.6818 - val_loss: 1.0436


--- Model 1 Evaluation ---

Test Accuracy: 0.6621

Test Loss:     1.0848
```

Classification Report:

	precision	recall	f1-score	support
airplane	0.80	0.62	0.70	1000
automobile	0.76	0.81	0.79	1000
bird	0.70	0.46	0.55	1000
cat	0.44	0.48	0.46	1000
deer	0.63	0.60	0.62	1000
dog	0.51	0.61	0.56	1000
frog	0.84	0.60	0.70	1000
horse	0.69	0.75	0.72	1000
ship	0.69	0.84	0.76	1000
truck	0.67	0.83	0.74	1000
accuracy			0.66	10000
macro avg	0.67	0.66	0.66	10000
weighted avg	0.67	0.66	0.66	10000

Confusion Matrix (10×10):

```
[[623  41  34  28  24  10   4  10 140  86]
 [  9 812   4  10   1   5   5   1  35 118]
 [ 48  16 455  92  94 141  35  58  34  27]
 [ 14  17  26 482  79 207  32  63  40  40]
 [ 20  12  34  86 602  73  25 100  30  18]
 [  9   8  31 177  35 613   8  74  23  22]
 [  4  26  25 140  59  75 603  16  28  24]
 [  8  10  22  39  51  63   3 753  14  37]
```

```
[ 35  32  11  15   4   6   1   9 845  42]
[  7  89   6  16   4   2   1  14  28 833]]
Saved: model1_training_curves.png
Saved: model1_confusion_matrix.png

======================================================================
MODEL 2: Improved CNN
======================================================================

Architecture:
  Conv2D(32,(3,3),same,relu) → BatchNorm → Conv2D(32,(3,3),same,relu) → BatchNorm → MaxPool(2,2) → Dropout(0.25)
  Conv2D(64,(3,3),same,relu) → BatchNorm → Conv2D(64,(3,3),same,relu) → BatchNorm → MaxPool(2,2) → Dropout(0.25)
  Flatten → Dense(256,relu) → BatchNorm → Dropout(0.5) → Dense(10,softmax)

Changes from Model 1:
  1. padding='same'      — Preserves spatial dims; prevents premature feature map collapse
  2. Double Conv blocks   — More complex feature learning before each pool step (VGG pattern)
  3. BatchNormalization   — Stabilizes gradients; mild regularization
  4. Dropout(0.25/0.5)    — Explicit regularization to combat overfitting
  5. Dense(256) vs 128    — Wider head for richer flattened input

Model: "Model2_Improved"
┏━━━
```

conv2d_2 (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
conv2d_3 (Conv2D)	(None, 32, 32, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 32)	128
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_4 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 64)	256

```
│ conv2d_5 (Conv2D)              │ (None, 16, 16, 64)       │        36,928 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────────┤
│ batch_normalization_3          │ (None, 16, 16, 64)       │           256 │
│ (BatchNormalization)           │                          │               │
├──────────────────────────────────────┼──────────────────────────────┼──────────────────┤
│ max_pooling2d_3 (MaxPooling2D) │ (None, 8, 8, 64)         │             0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────────┤
│ dropout_1 (Dropout)            │ (None, 8, 8, 64)         │             0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────────┤
│ flatten_1 (Flatten)            │ (None, 4096)             │             0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────────┤
│ dense_2 (Dense)                │ (None, 256)              │     1,048,832 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────────┤
│ batch_normalization_4          │ (None, 256)              │         1,024 │
│ (BatchNormalization)           │                          │               │
├──────────────────────────────────────┼──────────────────────────────┼──────────────────┤
│ dropout_2 (Dropout)            │ (None, 256)              │             0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────────┤
│ dense_3 (Dense)                │ (None, 10)               │         2,570 │
└──────────────────────────────────────┴──────────────────────────────┴──────────────────┘
```

Total params: 1,118,762 (4.27 MB)

Trainable params: 1,117,866 (4.26 MB)

Non-trainable params: 896 (3.50 KB)

Training Model 2 (epochs=10, batch_size=64, val_split=0.1)...

Epoch 1/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 28s 37ms/step - accuracy: 0.4864 - loss: 1.5123 - val_accuracy: 0.5824 - val_loss: 1.1891

Epoch 2/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 26s 37ms/step - accuracy: 0.6404 - loss: 1.0181 - val_accuracy: 0.6868 - val_loss: 0.9093

Epoch 3/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27s 38ms/step - accuracy: 0.6934 - loss: 0.8775 - val_accuracy: 0.6838 - val_loss: 0.9282

Epoch 4/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 26s 37ms/step - accuracy: 0.7256 - loss: 0.7843 - val_accuracy: 0.7264 - val_loss: 0.8111

Epoch 5/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27s 38ms/step - accuracy: 0.7460 - loss: 0.7212 - val_accuracy: 0.7666 - val_loss: 0.6699

Epoch 6/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27s 38ms/step - accuracy: 0.7656 - loss: 0.6640 - val_accuracy: 0.7356 - val_loss: 0.7682

Epoch 7/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 26s 37ms/step - accuracy: 0.7795 - loss: 0.6268 - val_accuracy: 0.7566 - val_loss: 0.7012

Epoch 8/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 26s 37ms/step - accuracy: 0.8000 - loss: 0.5658 - val_accuracy: 0.7634 - val_loss: 0.6954

Epoch 9/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27s 39ms/step - accuracy: 0.8109 - loss: 0.5320 - val_accuracy: 0.7558 - val_loss: 0.7370

Epoch 10/10

704/704 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 28s 39ms/step - accuracy: 0.8258 - loss: 0.4937 - val_accuracy: 0.7998 - val_loss: 0.6005

--- Model 2 Evaluation ---

Test Accuracy: 0.7837

Test Loss: 0.6404

Classification Report:

	precision	recall	f1-score	support
airplane	0.84	0.81	0.82	1000
automobile	0.92	0.87	0.89	1000
bird	0.83	0.60	0.70	1000
cat	0.62	0.54	0.58	1000
deer	0.78	0.74	0.76	1000
dog	0.59	0.81	0.68	1000
frog	0.86	0.79	0.83	1000
horse	0.78	0.87	0.82	1000
ship	0.84	0.91	0.88	1000
truck	0.85	0.90	0.87	1000
accuracy			0.78	10000
macro avg	0.79	0.78	0.78	10000
weighted avg	0.79	0.78	0.78	10000

Confusion Matrix (10×10):

```
[[808  12  25  19  10   8   3  16  64  35]
 [  9 870   2   6   0   5   2   1  21  84]
 [ 54   3 599  55  67  96  60  45  16   5]
 [ 16   5  28 543  49 267  30  34  15  13]
 [  9   1  28  47 738  50  29  83  14   1]
 [  6   1  10  99  23 809   3  45   1   3]
 [  5   2  17  75  32  57 793   5   9   5]
 [  7   1   9  18  22  68   0 870   1   4]
 [ 38  19   4   5   1   5   1   6 911  10]
 [ 14  36   1   7   1   3   1  13  28 896]]
Saved: model2_training_curves.png
Saved: model2_confusion_matrix.png

======================================================================
STEP 4: Sample Predictions — 20 Unseen Test Images
======================================================================
  Showing 20 test images (indices 0-19)
  Model 2 correct on these samples: 17/20
Saved: sample_predictions.png

======================================================================
STEP 5: Comparison Training Curves
======================================================================
Saved: comparison_training_curves.png

======================================================================
COMPARISON: Model 1 (Baseline) vs Model 2 (Improved)
======================================================================
```

Metric	Model 1 (Baseline)	Model 2 (Improved)
Test Accuracy	0.6621	0.7837
Test Loss	1.0848	0.6404
Input Shape	(32, 32, 3)	(32, 32, 3)
Conv Blocks	2 (single Conv/block)	2 (double Conv/block)
Padding	valid	same
BatchNormalization	No	Yes
Dropout	No	0.25 / 0.25 / 0.5
Dense Hidden Units	128	256
Epochs	10	10
Batch Size	64	64

Model 2 Improvement Rationale:

1. padding='same' — Preserves spatial dimensions before each pool step; prevents premature feature map collapse and allows stacking two Conv layers per block without size issues.
2. Double Conv blocks — Two consecutive Conv layers at the same filter depth before each MaxPool (VGG-style). More complex feature combinations learned per resolution level.
3. BatchNormalization — Normalizes activations within each mini-batch; stabilizes training and acts as mild regularization.
4. Dropout(0.25/0.5) — Explicit regularization. Model 1 showed overfitting (train acc >> val acc). Dropout forces redundant representations, improving generalization.

5. Wider Dense(256) — Larger classification head proportional to the richer 4096-dim flattened feature map from the deeper stack.

```
======================================================================
ANALYSIS COMPLETE
======================================================================
Output files generated:
  model1_training_curves.png
  model2_training_curves.png
  comparison_training_curves.png
  model1_confusion_matrix.png
  model2_confusion_matrix.png
  sample_predictions.png
```